Replay.

THE PROBLEM

You solved this bug three weeks ago. How?

Every developer wastes 30+ minutes per day re-finding solutions they've already discovered. The knowledge is buried in your terminal — you just can't find it.

🔍

Keyword search is broken

You grep through 10,000 lines of shell history using the wrong keywords. The fix was `docker build`, but you're searching for "container error."

📝

Notes go stale instantly

You told yourself "I'll remember this." You didn't. Notion docs are outdated, bookmarks are a graveyard, and Stack Overflow doesn't match your setup.

🔄

Same fix, fourth time

You've fixed this nginx SSL config three times already. Each time: 20 minutes of Googling, trial and error, and a prayer to the config gods.

🤖

AI has no memory

ChatGPT doesn't remember your last session. Your coding assistant doesn't know your project conventions. Every conversation starts from zero.

Replay.

THE SOLUTION

Semantic search over your terminal history

Not keyword matching — AI-powered understanding of what you did, when, and why.

```
~ zsh

$ replay search "how did I fix Docker"

Found 3 matches for "how did I fix Docker":

1. [87%] 3 weeks ago  /home/dev/api
  [exit:0] sudo docker build -t app .

2. [72%] yesterday    /home/dev/web
  [exit:0] docker compose up -d

3. [65%] 2 days ago   /home/dev
  [exit:1] docker build .  ← before the fix

$ replay explain "docker build -t app --no-cache ."

Builds a Docker image named "app" from the current
directory. The --no-cache flag forces a fresh build
by ignoring cached layers.
```

User Journey

1. **Install**
   `pip install replay-ai` — one command, zero config
2. **Index**
   `replay init` — reads bash/zsh/Atuin history, builds embeddings
3. **Search**
   `replay search "nginx SSL"` — natural language, semantic matching
4. **Discover**
   `replay fixes` — auto-detects bug-fix patterns you forgot about
5. **Understand**
   `replay explain <cmd>` — AI explains any command in plain English

Replay.

TECH STACK

Built with modern AI infrastructure

Privacy-first architecture. Your data never leaves your machine (except redacted embedding calls).

Data Pipeline

💻 **Capture**
Bash, Zsh, Atuin history readers

↓

⚙️ **Process**
Session clustering → chunking → fix detection → secret filter (16 patterns)

↓

🧠 **Embed**
Jina AI v3 / OpenAI embeddings with local SHA-512 fallback

↓

🔎 **Index & Search**
FAISS vector index, cosine similarity, threshold ranking

↓

💬 **AI Chat**
GPT-4o-mini for explain & summarize commands

Technologies

LANGUAGE
Python 3.10+
Type-hinted, dataclass-driven

CLI FRAMEWORK
Typer + Rich
12 commands, triple output

EMBEDDINGS
Jina AI v3
1024-dim, free tier

VECTOR STORE
FAISS
Local, no cloud dependency

LLM
GPT-4o-mini
Explain & summarize

SECURITY
16 Patterns
Secrets redacted pre-embed

160	12	<2s	$0
Tests passing	CLI commands	Query to result	Free with Jina

TARGET AUDIENCE

Who uses Replay?

Developers who live in the terminal and are tired of re-solving problems they've already cracked.

👨‍💻 Full-Stack Developers

Primary audience — daily terminal users

- Waste 20-30 min/day re-finding fixes
- Work across Docker, Git, K8s, databases
- Commands with complex flags and configs
- Value speed and privacy above all

🛠️ DevOps / SRE Engineers

High-value — incident-driven workflows

- "How did we fix this last time?"
- Complex deployment & infra commands
- Fix patterns are directly actionable
- Team knowledge sharing (v2)

🎓 Students & Junior Devs

Growth audience — learning accelerators

- `replay explain` turns commands into English
- Learn from their own past solutions
- Terminal becomes a knowledge base
- Free tier means zero cost to start

4.7M+
Developers on Atuin + bash/zsh

Terminal history is the **richest source of developer knowledge** that nobody indexes. Every command encodes what you were solving, what worked, what failed, and how you fixed it. Nobody else is doing semantic search over this.